Demo Video Script — WarehouseFlow Walkthrough

Structured 5-minute recording script for your portfolio.

Recording: OBS or QuickTime (whichever is available on Windows) **Setup:** Dashboard open fullscreen in Chrome at 1600x900, terminal ready, VS Code with engine.go visible

---

Pre-Recording Checklist

- [ ] Close Discord, email, notifications (Windows: Focus Assist → Priority only)
- [ ] Dashboard open at `dashboard/index.html`, everything paused
- [ ] Terminal at `DISTRIBUTED SYSTEMS/warehouseFlow/`
- [ ] VS Code open to `routing-service/router/engine.go`
- [ ] Mic test — audible without clipping

Scene 1 — The Problem (0:00–0:45)

[On camera, no screen]

> "Hey, I'm Nithish. I worked on Warehouse Management Systems at Manhattan Associates, and I kept running into questions in production that I couldn't answer cleanly with data. Questions like: when we lose a warehouse node at peak load, what actually happens? When a thousand orders compete for the last units of a popular SKU, can we guarantee we never oversell?"
>
> "So I built WarehouseFlow — a distributed order routing engine specifically designed to let me answer those questions empirically."

Scene 2 — Architecture (0:45–1:45)

[Screen: Dashboard showing warehouse map, nothing running]

> "This is the live operations view. An order comes in through the ingestion service, gets published to Kafka, then one of the horizontally-scaled routing tasks picks it up and decides which of three warehouse nodes to fulfill from."
>
> "Each warehouse has its own independent Redis — inventory, picker queue. Every routing decision is persisted to Postgres for audit. If routing fails, the order lands in an SQS dead-letter queue — never silently dropped."
>
> "Deployed on AWS via Terraform — ECS Fargate, MSK for Kafka, ElastiCache for Redis, RDS for Postgres."

Scene 3 — Normal Operation (1:45–2:30)

*[Click **START LOAD**]*

> "Let me start traffic — 3,000 orders a minute."
>
> *[Pause 3 seconds]*
>
> "Particles flow from the router to each warehouse, colored by destination. KPI strip shows throughput, P95, P99, success rate, DLQ depth, oversells."
>
> "Routing is region-aware — US-EAST orders prefer Warehouse A, US-WEST prefers C. Load distributes accordingly in the per-warehouse chart below."

Scene 4 — Failure Injection (2:30–3:30)

*[Click **CRASH WAREHOUSE B**]*

> "Now I simulate a real incident — Warehouse B's Redis dies."
>
> *[Pause 2 seconds]*
>
> "Warehouse B goes red, its circuit breaker flips to OPEN. Router stops sending traffic there. DLQ catches the orders that were in-flight at the exact moment of failure. Success rate dipped briefly, recovered. System auto-degraded to two warehouses."
>
> *[Pause 3 seconds, then click **RECOVER ALL**]*
>
> "Bring Warehouse B back. Circuit breaker goes HALF-OPEN, sends a probe, succeeds, goes back to CLOSED. DLQ drains as parked orders get reprocessed. Zero orders permanently lost."
>
> "This is the 'one crash equals capacity reduction, not outage' property that makes systems survivable in production."

Scene 5 — Key Experiment Finding (3:30–4:15)

*[Click **EXP 3 · CONTENTION**]*

> "The most counter-intuitive finding. I pre-loaded one warehouse with exactly 100 units of a popular SKU, then fired 1,000 concurrent orders for it."
>
> *[Let the autopilot run for 10 seconds]*
>
> "Tested with both concurrency strategies — optimistic compare-and-swap and pessimistic distributed locking. My hypothesis: only pessimistic would be correct; optimistic would oversell under heavy contention."
>
> "Actual result: both achieved zero oversells. The real correctness guarantee wasn't the client-side locking — it was the Lua atomic decrement inside Redis itself. Meaningful design insight: if your store supports atomic operations, client-side locking becomes a latency optimization, not a correctness requirement."

Scene 6 — Seven Experiments (4:15–4:45)

[Scroll to experiment button panel]

> "Seven experiments total:
>
> - Horizontal scaling — throughput vs ECS task count
> - Node failure resilience — what you just saw
> - Write contention under scarcity
> - CAP behavior under network partition
> - Kafka partition-to-consumer ratio
> - Noisy neighbor tail latency
> - Cold start after auto-scaling
>
> Each has raw data, charts, and written analysis comparing hypothesis to actual result."

Scene 7 — Close (4:45–5:00)

[On camera or dashboard wide]

> "Full code, seven experiment results, resilience patterns, and this dashboard are in the repo. Link below. Thanks."